

Lab: FHSS Multi-Channel Audio

Phân tích giấu tin FHSS trong âm thanh stereo đa kênh

1. Tổng quan bài lab

Bài lab mô phỏng một tình huống điều tra âm thanh số. Bạn được cung cấp một file stereo có khả năng chứa thông điệp ẩn theo ý tưởng FHSS. Thông điệp không được giấu trong cả hai kênh, mà chỉ nằm trong một kênh âm thanh. Nhiệm vụ của bạn là tìm đúng kênh bị chỉnh sửa, phân tích sự khác biệt, tìm khóa FHSS và khôi phục thông điệp.

1.1 Cơ sở lý thuyết

FHSS, viết tắt của **F**requency **H**opping **S**pread **S**pectrum, là kỹ thuật trải phổ trong đó vị trí truyền dữ liệu thay đổi theo một chuỗi giả ngẫu nhiên được sinh từ khóa. Ý tưởng chính là dữ liệu không nằm cố định tại một vị trí duy nhất, mà “nhảy” qua nhiều vị trí khác nhau. Trong bài toán giấu tin trên âm thanh, ý tưởng này có thể được mô phỏng bằng cách nhúng từng bit của thông điệp vào các sample khác nhau theo chuỗi vị trí do khóa tạo ra. Nếu không có khóa đúng, người phân tích sẽ khó biết cần đọc sample nào để khôi phục thông điệp.

Với âm thanh mono, toàn bộ tín hiệu chỉ có một kênh sample. Khi phân tích giấu tin, người học chủ yếu tập trung vào việc xác định các sample bị thay đổi và tìm đúng khóa FHSS. Tuy nhiên, với âm thanh stereo hoặc đa kênh, dữ liệu âm thanh không còn là một chuỗi sample đơn lẻ. Mỗi thời điểm âm thanh có thể gồm nhiều giá trị sample tương ứng với các kênh khác nhau, ví dụ kênh trái và kênh phải. Trong WAV PCM stereo, các sample của hai kênh thường được lưu xen kẽ: một sample của kênh trái, tiếp theo là một sample của kênh phải, rồi lặp lại.

Điểm khác biệt quan trọng của giấu tin FHSS trong âm thanh stereo là thông điệp có thể chỉ được nhúng vào một kênh nhất định. Nếu phân tích toàn bộ file stereo như một khối dữ liệu duy nhất, người phân tích có thể thấy có sự khác biệt, nhưng chưa biết sự khác biệt đó thuộc kênh nào. Vì vậy, trước khi khôi phục thông điệp, cần tách hoặc phân tích riêng từng kênh để xác định kênh nào có dấu hiệu bị chỉnh sửa.

Metadata của file âm thanh thường không đủ để phát hiện dữ liệu ẩn. Hai file có thể có cùng sample rate, cùng số kênh, cùng độ sâu bit và cùng thời lượng, nhưng vẫn khác nhau ở một số sample rất nhỏ bên trong. Vì vậy, quá trình điều tra cần đi từ mức tổng quát đến chi tiết: kiểm tra cấu trúc âm thanh, tách kênh, so sánh từng kênh với dữ liệu tham chiếu, rồi mới phân tích các sample bị thay đổi.

Trong mô hình giấu tin của bài này, mỗi bit được biểu diễn bằng một thay đổi nhỏ trên sample, chẳng hạn tăng hoặc giảm giá trị sample một lượng rất nhỏ. Những thay đổi này thường không làm thay đổi metadata và khó nhận ra bằng cách nghe thông thường. Tuy nhiên, khi so sánh với tín hiệu tham chiếu, các thay đổi này có thể được phát hiện. Nếu số sample bị thay đổi là NNN , có thể xem đó là NNN bit dữ liệu được nhúng. Với mã ASCII, 8 bit tạo thành 1 ký tự, nên số ký tự của thông điệp có thể được ước lượng bằng $N/8N / 8N/8$.

Khóa FHSS quyết định chuỗi vị trí sample được dùng để đọc lại các bit. Dùng sai khóa sẽ khiến chương trình đọc sai vị trí và tạo ra chuỗi ký tự vô nghĩa. Dùng đúng khóa sẽ đọc đúng chuỗi vị trí đã nhúng, từ đó khôi phục được thông điệp ban đầu. Vì vậy, trong bài toán stereo FHSS, người học phải giải quyết hai lớp vấn đề: xác định đúng kênh chứa dữ liệu ẩn và xác định đúng khóa để khôi phục thông điệp.

2. Môi trường và các file được cung cấp

Sau khi mở lab, bạn làm việc trong container fhss với người dùng ubuntu. Các lệnh trong tài liệu giả định bạn đang đứng tại thư mục /home/ubuntu. Nếu không chắc, hãy chạy `cd ~` trước mỗi nhóm lệnh.

File / công cụ	Vai trò
cover_stereo.wav	File stereo tham chiếu, chưa chứa dữ liệu ẩn.

stego_stereo.wav	File stereo nghi vấn, có thể chứa dữ liệu FHSS trong một kênh.
cover_left.wav	Kênh trái tham chiếu, dùng để so sánh với stego_left.wav.
cover_right.wav	Kênh phải tham chiếu, dùng để so sánh với stego_right.wav.
key_range.txt	Khoảng khóa nghi vấn dùng cho bước tìm khóa.
sample_change_extract.py	Công cụ tạo danh sách sample bị thay đổi sau khi đã xác định kênh nghi vấn.
key_search.py	Công cụ tìm khóa FHSS trong khoảng khóa nghi vấn.
recover_channel.py	Công cụ khôi phục thông điệp từ kênh đã xác định.

3. Quy trình thực hiện chi tiết

Bước 1 - Kiểm tra thông tin file âm thanh

Trước tiên, kiểm tra định dạng của các file WAV và lưu kết quả vào thư mục findings. Đây là bước kiểm kê ban đầu, giúp xác nhận file nào là stereo và file nào là mono.

```
cd ~
mkdir -p findings

file cover_stereo.wav stego_stereo.wav cover_left.wav cover_right.wav | tee findings/inspect.txt
soxi stego_stereo.wav | tee -a findings/inspect.txt
```

Lệnh file cho biết loại file và thông tin cơ bản như WAV, PCM, số bit và số kênh. Lệnh soxi cho thông tin chi tiết hơn như số kênh, sample rate, thời lượng và encoding.

Kết quả mong muốn là stego_stereo.wav có Channels: 2. Điều này xác nhận đây là file stereo cần được tách thành hai kênh riêng biệt trước khi phân tích.

Ý nghĩa: Nếu không kiểm tra metadata, bạn có thể phân tích sai loại file hoặc dùng nhầm công cụ. Bước này tạo bằng chứng trong findings/inspect.txt cho checkwork.

Bước 2 - Tách file stereo thành hai kênh mono

Dùng sox để tách file nghi vấn stego_stereo.wav thành hai file mono: stego_left.wav và stego_right.wav.

```
sox stego_stereo.wav stego_left.wav remix 1
sox stego_stereo.wav stego_right.wav remix 2

soxi stego_left.wav | tee findings/split.txt
soxi stego_right.wav | tee -a findings/split.txt
```

Trong sox, remix 1 lấy kênh thứ nhất, tức kênh trái. remix 2 lấy kênh thứ hai, tức kênh phải. Sau khi tách, dùng soxi kiểm tra lại để chắc chắn hai file đầu ra là file mono hợp lệ.

Không cần ghi thủ công SPLIT_DONE. Việc split được chứng minh bằng chính output thật của soxi trong findings/split.txt.

Bước 3 - So sánh từng kênh với kênh tham chiếu

Sau khi có hai kênh mono, so sánh từng kênh stego với kênh cover tương ứng. Kênh nào có khác biệt là kênh nghi vấn.

```
mkdir -p findings/diffs

cmp -l cover_left.wav stego_left.wav > findings/diffs/left.cmp.txt || true
```

```
cmp -l cover_right.wav stego_right.wav > findings/diffs/right.cmp.txt || true
```

```
wc -l findings/diffs/left.cmp.txt findings/diffs/right.cmp.txt | tee findings/channel_diff.txt  
cat findings/channel_diff.txt
```

Lệnh `cmp -l` liệt kê các byte khác nhau giữa hai file. Nếu hai file giống hệt nhau, file kết quả sẽ có 0 dòng. Nếu có khác biệt, mỗi dòng tương ứng một byte khác nhau. Lệnh `wc -l` đếm số dòng khác biệt của từng kênh.

Ví dụ kết quả có thể là:

```
0 findings/diffs/left.cmp.txt  
73 findings/diffs/right.cmp.txt  
73 total
```

Kết quả này cho thấy kênh trái không bị thay đổi, còn kênh phải có khác biệt. Vì vậy, trong báo cáo cuối cùng, `HIDDEN_CHANNEL` nên được điền là `right`.

Lưu ý: Số byte khác biệt không nhất thiết bằng số sample thay đổi. WAV PCM 16-bit dùng 2 byte cho mỗi sample, nên một sample thay đổi có thể làm thay đổi 1 hoặc 2 byte.

Bước 4 - Phân tích các sample bị thay đổi

Sau bước 3, kênh phải là kênh nghi vấn. Phân tích sâu hơn ở mức sample để biết có bao nhiêu sample bị chỉnh sửa.

```
./sample_change_extract.py --cover cover_right.wav --input stego_right.wav
```

```
wc -l findings/changed_samples.txt  
head findings/changed_samples.txt
```

Script `sample_change_extract.py` đọc từng sample của `cover_right.wav` và `stego_right.wav`, sau đó ghi các vị trí khác nhau vào `findings/changed_samples.txt`. Mỗi dòng có dạng:

```
sample_index diff
```

Trong đó `sample_index` là vị trí sample bị thay đổi, còn `diff` là độ lệch giữa sample trong file stego và sample trong file cover.

Nếu `wc -l` trả về 72, nghĩa là có 72 sample bị thay đổi. Với mô hình của bài này, mỗi sample thay đổi tương ứng một bit. Vì 8 bit tạo thành 1 ký tự ASCII, số ký tự của thông điệp là $72 / 8 = 9$. Do đó, trong `report.txt`, `MSG_LEN` nên là 9 và `CHANGED_SAMPLES` nên là 72.

Bước 5 - Quan sát phân bố thay đổi bằng awk, sort, uniq

Bước này không chỉ để lấy điểm checkwork mà giúp bạn hiểu dữ liệu thay đổi có cấu trúc. Các thay đổi nhỏ như -2 và 2 cho thấy dữ liệu được nhúng bằng cách chỉnh nhẹ sample.

```
awk '{print $2}' findings/changed_samples.txt | sort | uniq -c  
awk '{print int($1/1024)}' findings/changed_samples.txt | sort | uniq | wc -l
```

Lệnh đầu tiên đếm các giá trị `diff`. Lệnh thứ hai ánh xạ `sample_index` về frame bằng cách chia cho 1024. Kết quả này củng cố rằng frame size dùng trong bài là 1024.

Ý nghĩa: Các lệnh `awk` giúp bài lab không chỉ dựa vào Python. Bạn đang dùng công cụ dòng lệnh để kiểm tra cấu trúc dữ liệu đã bị chỉnh sửa.

Bước 6 - Tìm khóa FHSS

Khóa FHSS không được cho trực tiếp. File `key_range.txt` cho biết khoảng khóa nghi vấn cần kiểm tra. Chạy lệnh sau để xem khoảng khóa:

```
cat key_range.txt
```

Sau đó dùng `key_search.py` để thử các khóa trong khoảng đó:

```
./key_search.py --cover cover_right.wav --input stego_right.wav --start 24000 --end 25000 --frame-size 1024 --msg-len 9
```

```
cat findings/key_candidates.txt
```

Công cụ key_search.py không in thông điệp đầy đủ. Nó chỉ hiển thị các key ứng viên, điểm số và preview. Key tốt nhất là key tạo ra chuỗi có dạng hợp lý, ví dụ bắt đầu bằng CHAN.

Kết quả mong muốn có dạng:

```
rank=1 score=48 key=24680 preview=CHAN????? candidate=LIKELY
```

Từ đó có thể xác định khóa phù hợp là 24680. Giá trị này sẽ được dùng ở bước recover và điền vào mục KEY trong report.txt.

Bước 7 - Khôi phục thông điệp

Dùng kênh phải, frame size, độ dài thông điệp và key tìm được để khôi phục thông điệp.

```
./recover_channel.py --cover cover_right.wav --input stego_right.wav --output recovered.txt --log recover.log --key 24680 --frame-size 1024 --msg-len 9
```

```
cat recovered.txt
```

Nếu các bước trước đúng, recovered.txt sẽ chứa:

```
CHANNEL42
```

Đây là thông điệp đã được giấu trong kênh phải của file stereo.

Bước 8 - Tạo và điền report.txt

```
nano report.txt
```

Nội dung report.txt nên có dạng:

```
LEFT_DIFF_BYTES=0
RIGHT_DIFF_BYTES=73
HIDDEN_CHANNEL=right
CHANGED_SAMPLES=72
FRAME_SIZE=1024
MSG_LEN=9
KEY=24680
MESSAGE=CHANNEL42
```

Giải thích từng mục trong report.txt

LEFT_DIFF_BYTES: Số byte khác biệt giữa cover_left.wav và stego_left.wav. Lấy từ kết quả wc -l ở bước so sánh kênh. Nếu bằng 0, kênh trái không bị chỉnh sửa.

RIGHT_DIFF_BYTES: Số byte khác biệt giữa cover_right.wav và stego_right.wav.

HIDDEN_CHANNEL: Kênh được kết luận là chứa dữ liệu FHSS. Nếu left diff bằng 0 và right diff khác 0, điền right.

CHANGED_SAMPLES: Số dòng trong findings/changed_samples.txt. Lấy bằng wc -l. Đây là số sample âm thanh bị thay đổi.

FRAME_SIZE: Kích thước frame dùng trong phân tích FHSS. Trong bài này dùng 1024, thể hiện ở bước ánh xạ sample_index về frame.

MSG_LEN: Độ dài thông điệp theo ký tự. Suy ra từ CHANGED_SAMPLES / 8. Nếu có 72 sample thay đổi thì MSG_LEN = 9.

KEY: Khóa FHSS tìm được từ findings/key_candidates.txt.

MESSAGE: Thông điệp khôi phục được trong recovered.txt.

4. Kiểm tra kết quả bằng checkwork

Sau khi hoàn thành các bước và tạo report.txt, quay về terminal host và chạy:

```
checkwork
```